



Lab 9. Continuous Testing

By the end of this lab exercise, you should be able to:

- Analyze code coverage with JaCoCo
- Improve the code coverage
- Set up SonarQube to automatically scan your repository
- Create a project gating system to automatically decide whether the code is safe to deploy
- Add integration and acceptance tests with Docker Compose
- Incorporate all of the steps into the Jenkins pipeline

Code Coverage with Jacoco

Create a new Jenkins project which will download the following sample repository and run unit tests with a Jacoco plugin for Java and display code coverage reports.

Fork [the maven-examples repository](#) to your GitHub account. Be sure to *uncheck* the **Copy the master branch only** option, as follows:

Create a new fork

A *fork* is a copy of a repository. Forking a repository allows you to freely experiment with changes without affecting the original project. [View existing forks.](#)

Owner *

eeganlf

Repository name *

maven-examples

By default, forks are named the same as their upstream repository. You can customize the name to distinguish it further.

Description (optional)

☐ Copy the `master` branch only

Contribute back to lfraining/maven-examples by adding your own branch. [Learn more.](#)

ⓘ You are creating a fork in your personal account.

Create fork

Install the **Coverage** Plugin for Jenkins.

Create a Maven project named **maven-examples**, provide the repository you forked as source, add Maven goals to run tests on it. Use “clean test” as the Maven goal.

Under the **Source Code Management** section, select **Git** and paste the repository URL of the **maven-examples** repository that you forked.

Dashboard > maven-examples > Configuration

Configure

- General
- Source Code Management**
- Build Triggers
- Build Environment
- Pre Steps
- Build
- Post Steps
- Build Settings
- Post-build Actions

Source Code Management

☐ None

☒ Git ?

Repositories ?

Repository URL ?

https://github.com/eeganlf/maven-examples.git

Credentials ?

- none -

+ Add

Advanced

Under the **Build** section, enter the path to the `pom.xml` as `code-coverage-jacoco/pom.xml`, and type `clean test` into the **Goals** text box.

Configure

- General
- Source Code Management
- Build Triggers
- Build Environment
- Pre Steps
- Build**

Build

Root POM ?
code-coverage-jacoco/pom.xml

Goals and options ?
clean test

Advanced ▾

From post-build actions, choose **Record code coverage results**.

Dashboard > Instavote > maven-examples > Configuration

Configure

- General
- Source Code Management
- Triggers
- Environment
- Pre Steps
- Build**
- Post Steps
- Build Settings
- Post-build Actions

Build

Root POM ?
code-coverage-jacoco/pom.xml

Goals and options ?
clean test

Filter

- Aggregate downstream test results
- Archive the artifacts
- Build other projects
- Deploy artifacts to Maven repository
- Discover reference build
- Mine SCM repository
- Publish HTML reports
- Record code coverage results**
- Record fingerprints of files to track usage
- Stop Docker Containers
- Git Publisher
- SonarQube analysis with Maven
- Build other projects (manual step)
- Editable Email Notification
- Set GitHub commit status (universal)
- Set build status on GitHub commit [deprecated]
- Slack Notifications
- Trigger parameterized build on other projects
- Delete workspace when build is done

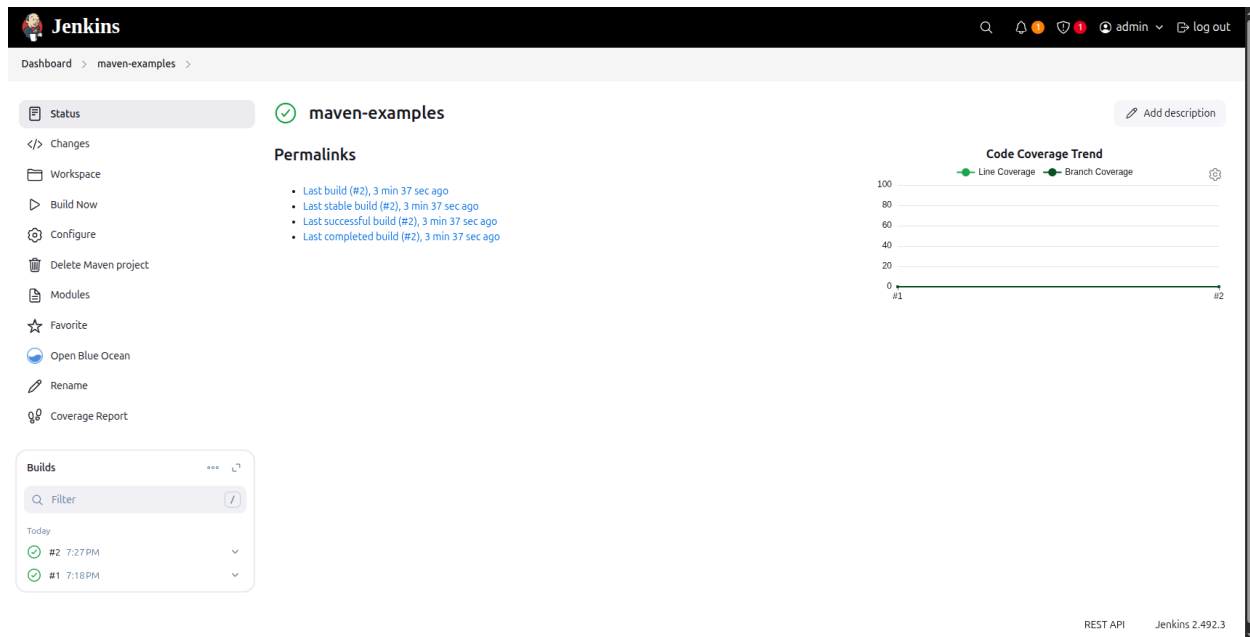
Add post-build action ▾

Save Apply

REST API Jenkins 2.492.3

Select **JaCoCo Coverage Reports** as the **Coverage Parser** and **Save**.

Build the project twice and refresh the page after the build is complete:



You should see a Code Coverage Trend between the two builds. It won't show much at the moment because you have only built the project once and there are no tests. You will add tests to the master branch next.

Create and merge a pull request from the `ut1` branch in your repository onto your `master` branch.

First, navigate to **Pull requests**:

Search or jump to... Pull requests Issues Codespaces Marketplace Explore

eeganlf / maven-examples Public
forked from lftraining/maven-examples

<> Code **Pull requests** Actions Projects Wiki Security Insights Settings

master 1 branch 0 tags Go to file Add file <> Code

This branch is up to date with lftraining/maven-examples:master. Contribute Sync fork

eeganlf removed test 8eb1394 2 hours ago 63 commits

assembly-plugin	Move license to the root directory	7 years ago
code-coverage-jacoco	removed test	2 hours ago
findbugs-cookbook	First version of FindBugs Cookbook	9 years ago

About
No description, website, or topics provided.
Readme View license 0 stars 0 watching 669 forks

Click **New pull request**:

eeganlf / maven-examples Public
forked from lftraining/maven-examples

<> Code **Pull requests** Actions Projects Wiki Security Insights Settings

Filters is:pr is:open Labels 9 Milestones 0 **New pull request**

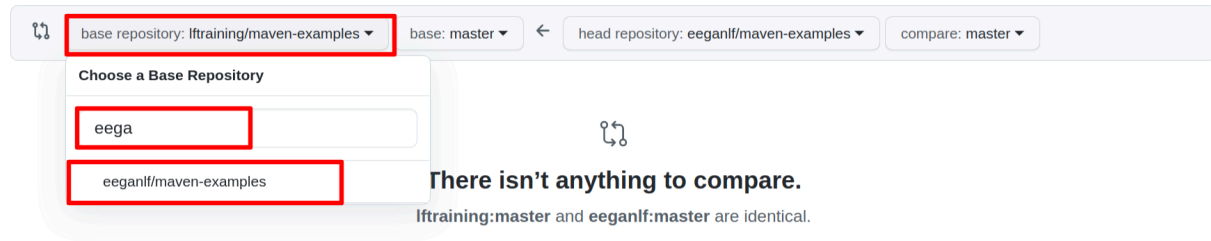
Welcome to pull requests!

Pull requests help you collaborate on code with other people. As pull requests are created, they'll appear here in a searchable and filterable list. To get started, you should [create a pull request](#).

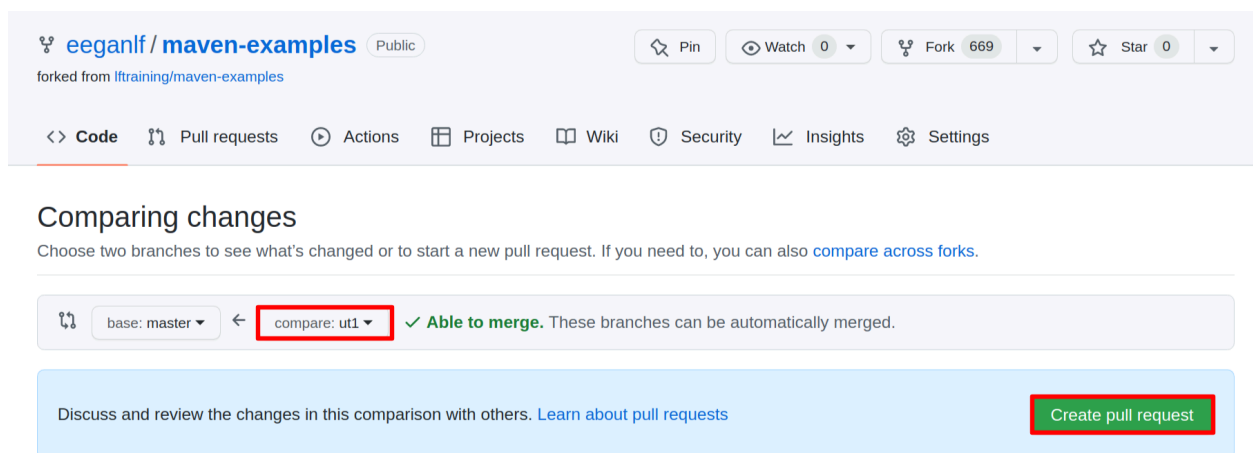
Click on the **base repository** dropdown and search for your GitHub username. When you find your repository, select it as the base repository:

Comparing changes

Choose two branches to see what's changed or to start a new pull request. If you need to, you can also [compare across forks](#).



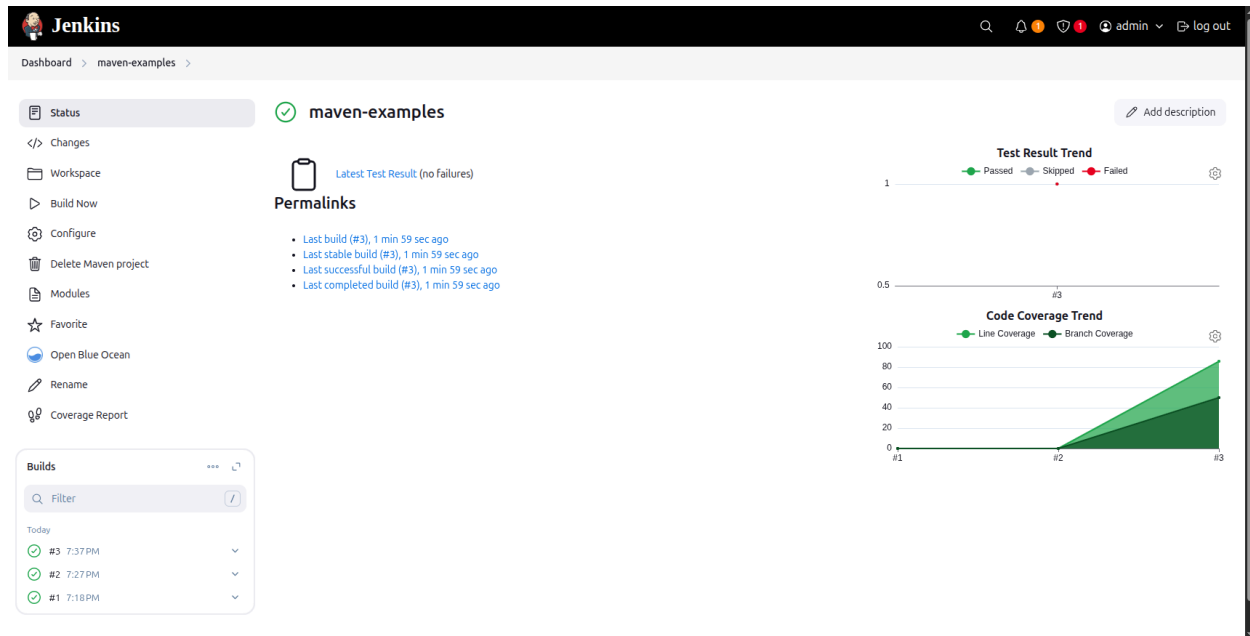
Select the **ut1** branch as the branch to **compare** and click **Create pull request**:



Approve the request as normal.

This will add pre-written tests to your master branch so that Jacoco will show some coverage in Jenkins.

Go back to the Maven project page you created earlier and build the project; refresh the page when the build is finished and you should see a code coverage report get published on the project page.



Adding Static Code Analysis with SonarQube

Steps:

- Set up Sonar Cloud.
- Install SonarQube Scanner plugin for Jenkins.
- Configure SonarQube Servers from Jenkins Systems Configuration.

Set Up Sonar Cloud as SonarQube Server

Head over to [SonarCloud](https://sonarcloud.io) and sign up using your GitHub account.

[WEBINAR] SCALING CLEAN CODE ACROSS YOUR ENTERPRISE - NOVEMBER 15 [Register Now](#)

 [Solutions](#) [Products](#) [Company](#) [Resources](#) [Knowledge](#) [FREE SONAR](#) [EXPLORE PRICING](#)

sonarcloud  | [Features](#) [What's New](#) [Roadmap](#) [Pricing](#) [SIGN UP](#) [LOG IN](#)

AS A SERVICE. SONARCLOUD.

clean code in your cloud workflow with {SonarCloud}

Enable your team to deliver clean code consistently and efficiently with a tool that easily integrates into the cloud DevOps platforms and extend your CI/CD workflow.



Hey 🙌 thanks for stopping by! What brought you to our site today?





Try SonarCloud for free

1-CLICK SIGNUP WITH:

 **GITHUB**

 **BITBUCKET**

 **GITLAB**

 **AZURE DEVOPS**

By logging in, you're agreeing to our [Terms of Service](#).

YOUR TEAM COMPANION FOR CLEAN CODE

**free**
for open-source projects

**fast**
project onboarding and analysis

**precise**
and actionable results

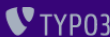
TRUSTED BY

 **Microsoft**

 **APACHE**
FUNDATION

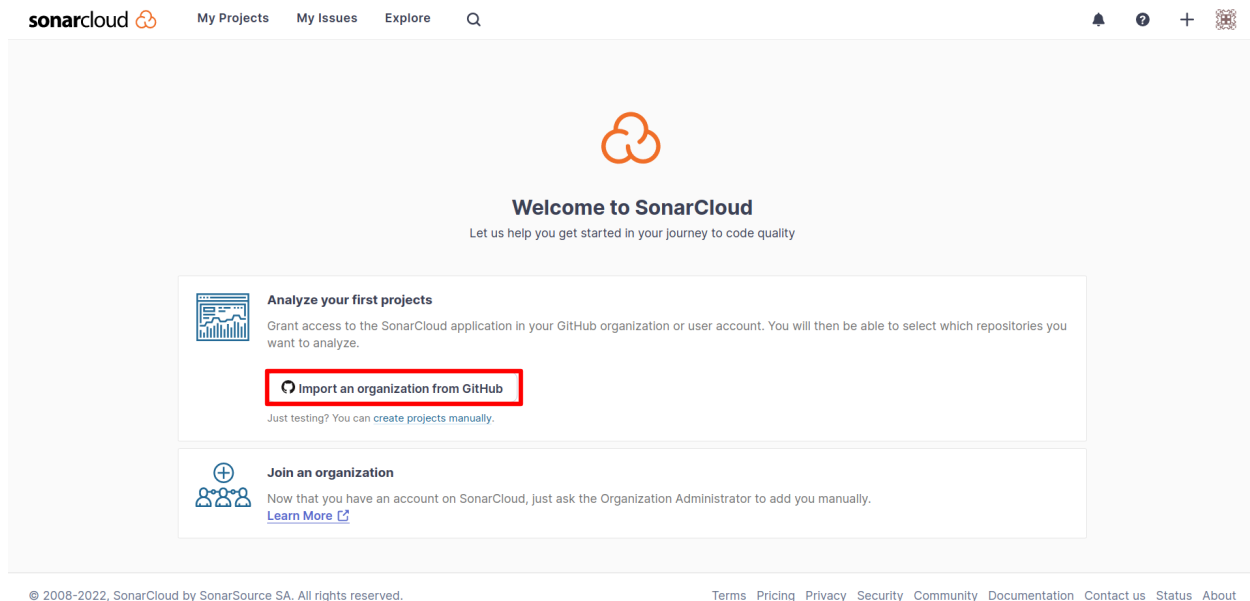
 **WIKIMEDIA**
FOUNDATION

 **brave**

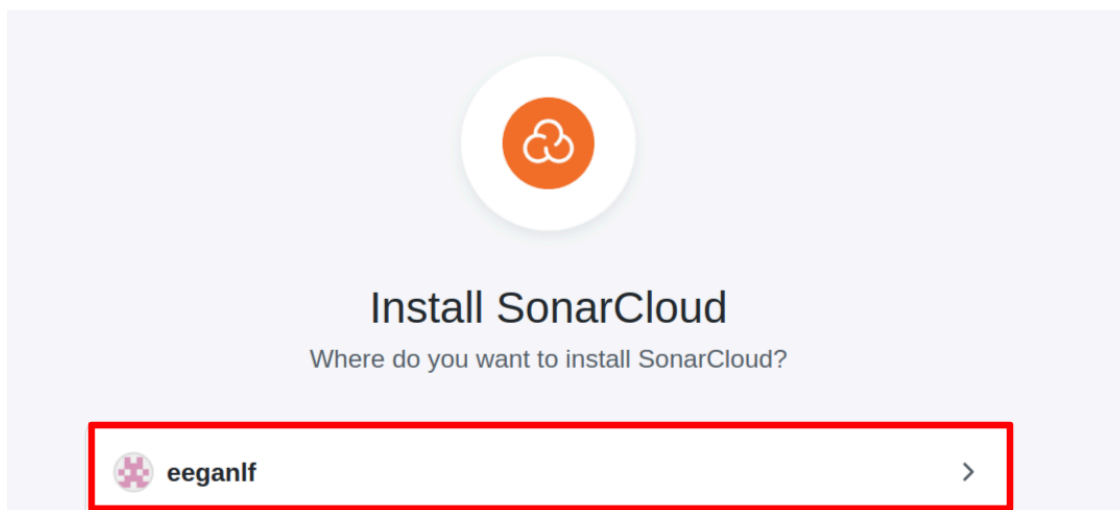
 **TYP03**

Ensure that you are logged into the GitHub account from the same browser in order to provide authorization. Follow the prompts to authorize SonarCloud with GitHub.

From the welcome page, click **Import an organization from GitHub**.



Choose your GitHub organization.



Provide access to all your repositories:

Install SonarCloud

Install on your personal account eeganlf

☒ **All repositories**

This applies to all current *and* future repositories owned by the resource owner.
Also includes public repositories (read-only).

☐ **Only select repositories**

Select at least one repository.
Also includes public repositories (read-only).

with these permissions:

✓ **Read** access to code and metadata

✓ **Read** and **write** access to checks, commit statuses, pull requests,
and security events

User permissions

SonarCloud can also request users' permission to the following resources. These
permissions will be requested and authorized on an individual-user basis.

✓ **Read** access to email addresses

Install

Cancel

Next: you'll be directed to the GitHub App's site to complete setup.

You will be redirected back to SonarCloud.

Provide an organization name to be created on SonarCloud. This is important, as you are going to use it as a reference later when you integrate it with Jenkins. Your GitHub username should be auto-populated. If not, enter it into the **Key** box. Click **Continue**.

Create an organization

An organization is a space where a team or a whole company can collaborate across many projects.

1

Import organization details

Import  eeganlf into a SonarCloud organization ✕

Key * ?

eeganlf

Organization key must start with a lowercase letter or number, followed by lowercase letters, numbers or hyphens, and must end with a letter or number. Maximum length: 255 characters.

[Add additional info](#) ▼

i

All members from your GitHub organization eeganlf will be added to your SonarCloud organization. As they connect to SonarCloud with their GitHub account, members will automatically have access to your SonarCloud organization and its projects.[See all members on GitHub](#) ↗

Continue

Since your repository is public, you can proceed with the free plan.

 SonarQube
cloud

My Projects My Issues Explore

🔍 ⚙️ 👤 + 🔒

2

Choose a plan

Free

For individual developers, students and open-source projects

\$0

Select Free

What's included

- Up to 50k private lines of code
- Up to 5 members
- Unlimited public lines of code

Key features

- Open-source and private projects
- 30 languages supported
- Issue detection and SAST
- Analyze the main branch & pull requests
- DevOps platform integration

Team Free trial available

Essential capabilities for small teams and business

Starting at
\$64 \$32/month - 50% off

Start Free Trial

What's included

- Up to 1.9M private lines of code
- Unlimited members
- Unlimited public lines of code

Everything in Free and

- 14-day free trial, cancel any time
- Open-source and private projects
- Analyze feature branches, maintenance branches, & pull requests
- Define the quality standard for your team
- Advance issue detection
- Deeper SAST
- Synchronized user management

Enterprise

Mission critical flexibility, scalability, and performance.

To get started
Talk to sales

Contact sales

What's included

- From 5M private lines of code
- Unlimited members
- Unlimited public lines of code

Everything in Team and

- Enterprise-Level Hierarchy
- Secure SAML single sign-on (SSO)
- Management reporting
- Centralized enterprise billing
- Enterprise-grade security
- AWS Marketplace transaction option
- 6 additional enterprise languages: ABAP, APEX, COBOL, JCL, PL/I, RPG

Available packs

- Premium Support

Create Organization

Choose the repositories that you want to analyze with SonarQube. Select **maven-examples** and the **example-voting-app**. Click **Set Up**:

Analyze projects - Select repositories

Organization *
eeganlf eeganlf [Import another organization](#)

☐ Select all available repositories

- ☐ argocd-example-apps
- ☐ armory-minnaker
- ☐ devops-repo
- ☐ helm-charts-lfs269-original
- ☐ json-server
- ☒ LFS261-example-voting-app
- ☐ LFS269-helm-charts
- ☐ localtunnel
- ☒ maven-examples

2 repositories selected
2 repositories will be created as public projects on SonarCloud
[Set Up](#)

If you are not redirected to your **My Projects** page in SonarCloud, navigate there now via the **My Projects** link.

sonarcloud [My Projects](#) [My Issues](#) [Explore](#)

Set up 2 projects for Clean as You Code

The new code definition sets which part of your code will be considered new code.

This helps you focus attention on the most recent changes to your project, enabling you to follow the Clean as You Code methodology.

Learn more: [New Code Definition](#)

Set a new code definition for your organisation to use it by default for all new projects
This can help you use the Clean as You Code methodology consistently across projects.
[eeganlf - Administration - New Code](#)

This is the page that lists all of your projects.

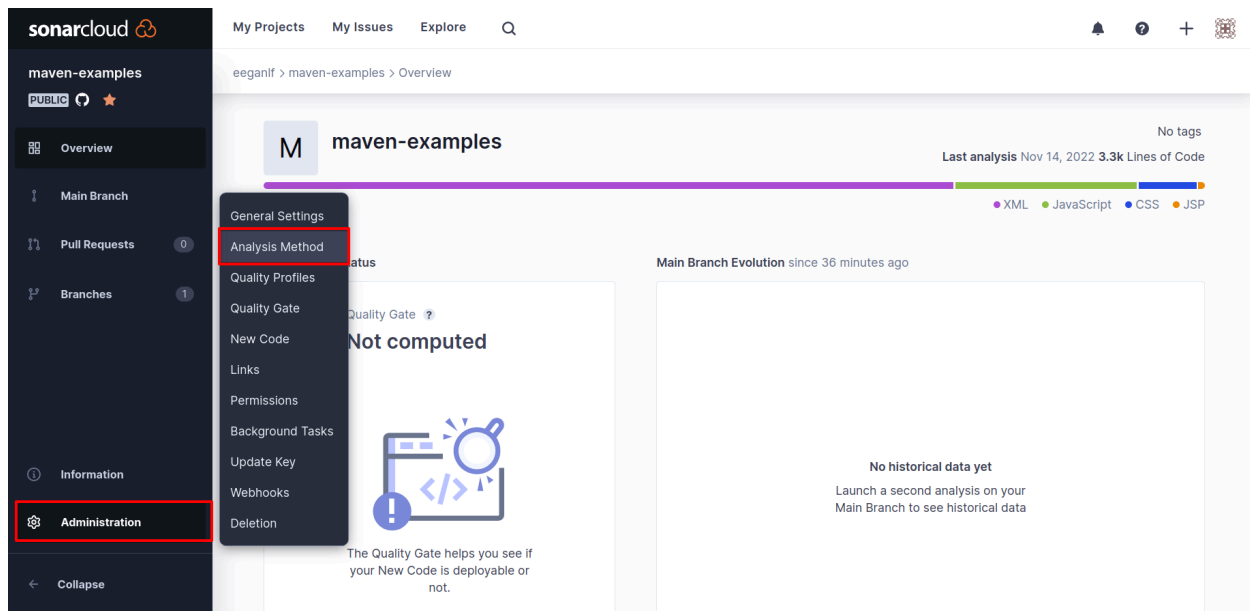
The screenshot shows the SonarCloud interface. On the left, there are filters for Quality Gate (Passed: 0, Failed: 0), Reliability (A: 0, B: 0, C: 0, D: 1, E: 0), and Security (A: 1, B: 0). The main area shows a list of projects. The first project is 'eeganlf / LFS261-example-voting-app' with status 'NEW' and 'PUBLIC'. Below it is 'eeganlf / maven-examples' with status 'NEW' and 'PUBLIC'. The 'maven-examples' project has a 'Not computed' status. The page also shows filters for Quality Gate, Reliability, and Security.

Click on the **maven-examples** project:

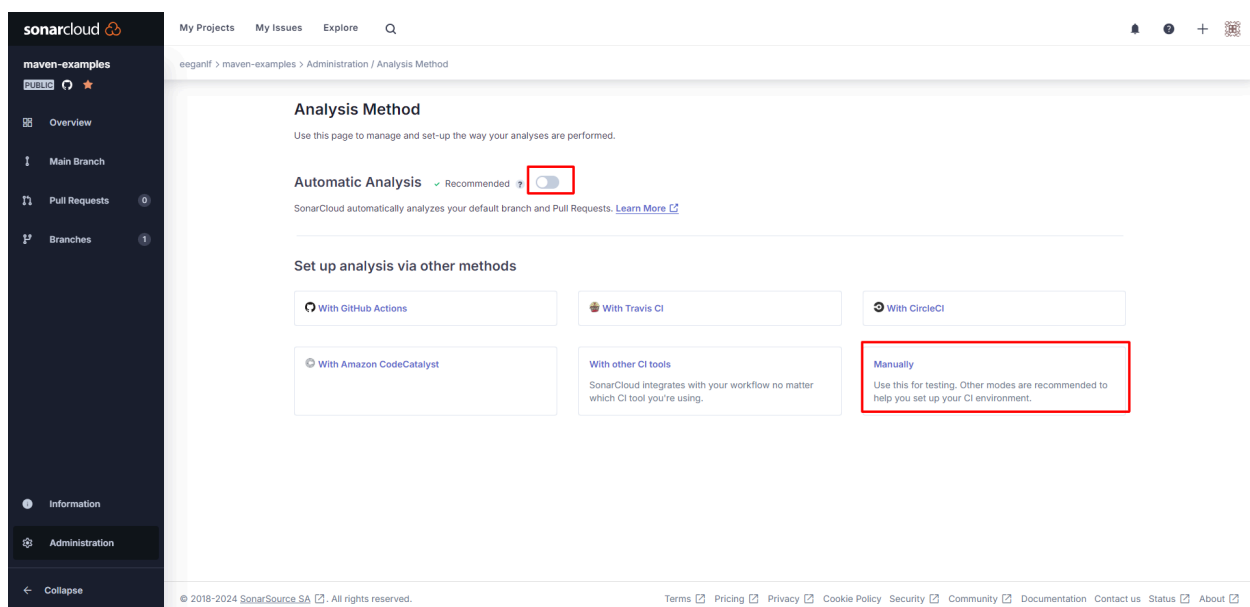
The screenshot shows the SonarCloud project page for 'eeganlf / maven-examples'. The project is marked as 'NEW' and 'PUBLIC'. The status is 'Not computed'. The last analysis was on 11/14/2022 at 8:14 PM, showing 3.3k Lines of Code in XML, JavaScript, etc. The metrics are: Bugs (D 5), Vulnerabilities (A 0), Hotspots Reviewed (A 100%), Code Smells (A 92), and Duplications (0.0%).

You will be taken to the **maven-examples** project page. You must disable automatic analysis as you want Jenkins to tell SonarCloud when to analyze your code.

You must also configure your project. To accomplish this, click navigate to **Administration > Analysis Method**.



Uncheck **Automatic Analysis** and click **Set up analysis via other methods > Manually**:



Under **What option best describes your build?** select **Other**. Select **Linux** for the operating system.

Analyze from your local sources

Run analysis on your project

What option best describes your build?

Maven Gradle .NET C, C++ or ObjC **Other (for JS, TS, Go, Python, PHP, ...)**

Which OS do you run your build on?

Linux Windows macOS

Download and unzip the SonarScanner for Linux

And add the `bin` directory to the `PATH` environment variable

[Download](#)

Configure the SONAR_TOKEN environment variable

- 1 Name of the environment variable: `SONAR_TOKEN`
- 2 Value of the environment variable: `F21b498221795bb506ac8b21aee7f6c75acce1ef`

Execute the SonarScanner from your computer

Run the following command in your project's folder.

```
sonar -scanner \
  -Dsonar.organization=eeganelf \
  -Dsonar.projectKey=eeganelf_maven-examples \
  -Dsonar.sources=. \
  -Dsonar.host.url=https://sonarcloud.io
```

[Copy](#)

Please visit the [SonarScanner documentation](#) for more details.

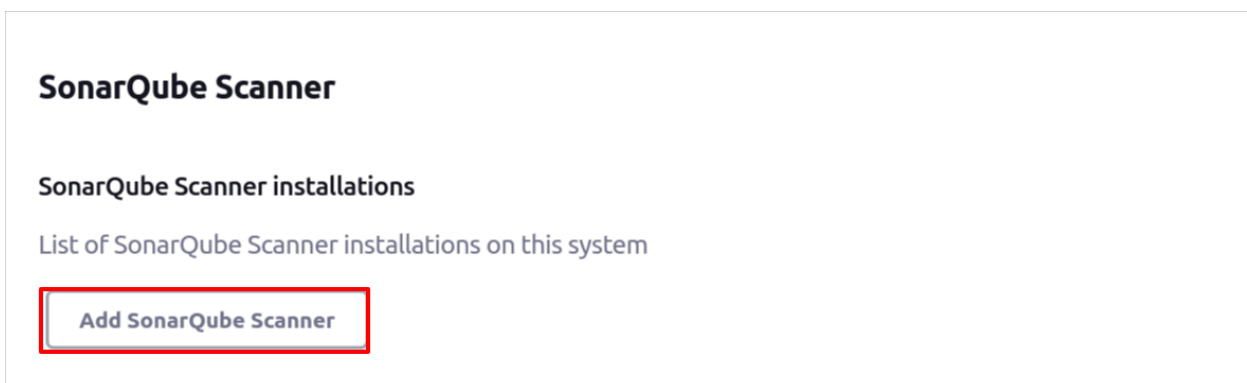
Notice that a command is generated. This command contains configuration properties that we will use to integrate SonarQube with Jenkins.

Integrate SonarQube with Jenkins

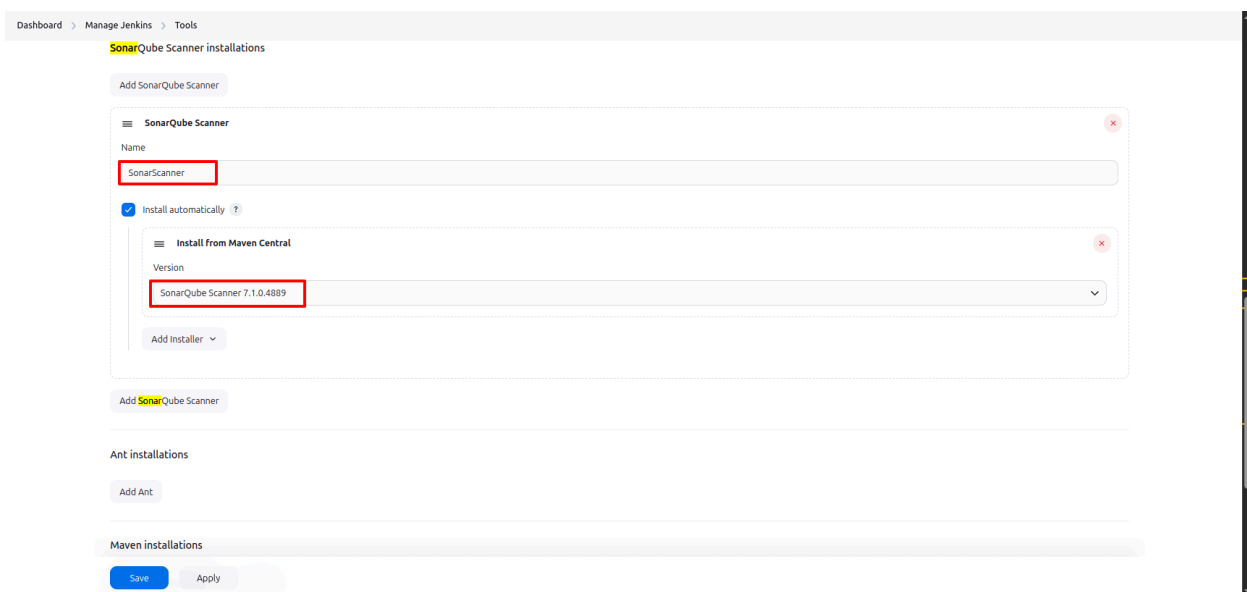
Install the SonarQube Scanner plugin for Jenkins.

Install	Name ↓	Released
☒	SonarQube Scanner 2.17.2 External Site/Tool Integrations Build Reports This plugin allows an easy integration of SonarQube , the open source platform for Continuous Inspection of code quality.	5 mo 13 days ago

From **Jenkins > Manage Jenkins > Tools > SonarQube Scanner** click **Add SonarQube Scanner**:



Name the SonarQube Scanner **SonarScanner** which will be referred to later when you write the pipeline code. Also, choose whichever is the latest version, which should be automatically selected from the dropdown menu:



From the Jenkins home, navigate to **Manage Jenkins > Security > Credentials**:

[Manage Jenkins](#) > [Credentials](#)

T	P	Store ↓	Domain	ID	Name
		System	(global)	0d47ee8b-691f-4e9e-8b9e-02049e396a56	Secret text
		System	(global)	jenkins-github-access-token	jenkins-github-access-token
		System	(global)	github-auth-token	eeganlf/***** (auth token for github)

Stores scoped to Jenkins

P	Store ↓	Domains
	System	(global)

Click **Global credentials (unrestricted)**:

System [+ Add domain](#)

Domain ↓	Description
Global credentials (unrestricted)	Credentials that should be available irrespective of domain specification to requirements matching.

Icon: ☐ S ☐ M ☒ L

Click **Add Credentials**:

Global credentials (unrestricted)

[+ Add Credentials](#)

Credentials that should be available irrespective of domain specification to requirements matching.

ID	Name	Kind	Description
 0d47ee8b-691f-4e9e-8b9e-02049e396a56	Secret text	Secret text	
 jenkins-github-access-token	jenkins-github-access-token	Secret text	
 github-auth-token	eeganlf/***** (auth token for github)	Username with password	auth token for github 

Icon: S M **L**

Add new credentials with:

- Kind “Secret text”.
- Copy the value for the **SONAR_TOKEN** found on the configuration page on SonarCloud for the **maven-examples** project:

elf > maven-examples > Configure

- 1 Name of the environment variable: SONAR_TOKEN
- 2 Value of the environment variable: f21b498221705bb506ac8b21aee7f6c75acce1ef

Execute the SonarScanner from your computer

Run the following command in your project's folder.

```
sonar-scanner \
-Dsonar.organization=eeganlf \
-Dsonar.projectKey=eeganlf_maven-examples \
-Dsonar.sources=. \
-Dsonar.host.url=https://sonarcloud.io
```

[Copy](#)

Please visit the [SonarScanner documentation](#) for more details.

- Paste it in the “Secret” field on Jenkins.
- Add ID and Description as **sonar-maven-examples**.

Refer to the following screenshot while adding credentials:

Dashboard > Manage Jenkins > Credentials > System > Global credentials (unrestricted)

New credentials

Kind
Secret text

Scope ?
Global (Jenkins, nodes, items, all child items, etc)

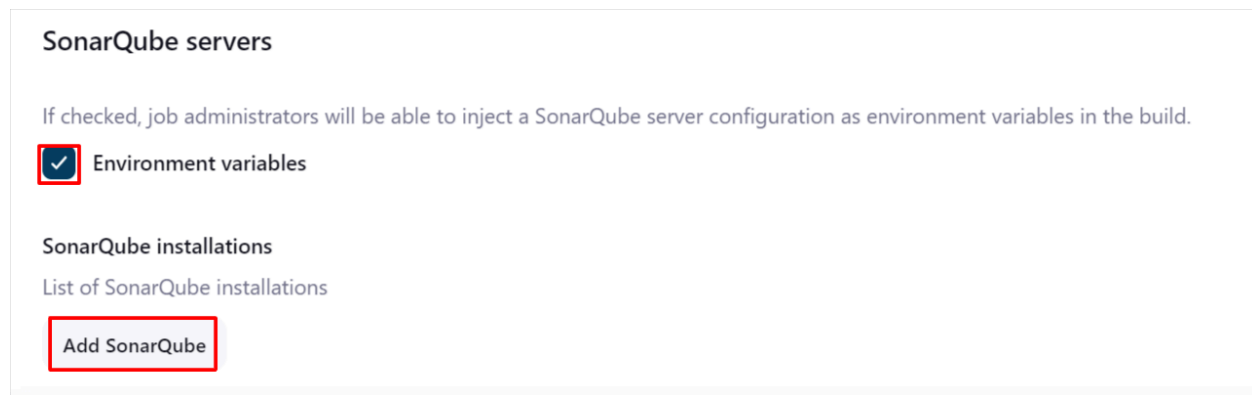
Secret

ID ?
sonar-maven-examples

Description ?
sonar-maven-examples

[Create](#)

Now navigate to **Jenkins > Manage Jenkins > System > SonarQube Servers** and add a new entry.

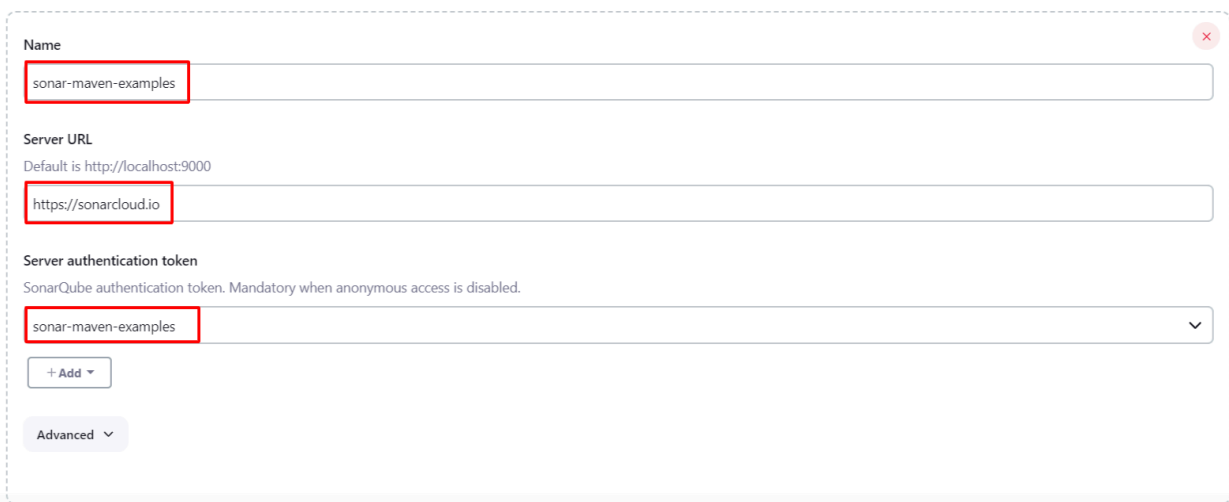


The screenshot shows the 'SonarQube servers' configuration page. At the top, there is a note: 'If checked, job administrators will be able to inject a SonarQube server configuration as environment variables in the build.' Below this, the 'Environment variables' checkbox is checked and highlighted with a red box. Under the 'SonarQube installations' section, there is a list of installations and an 'Add SonarQube' button, which is also highlighted with a red box.

Provide the configuration as shown below:

- Check the **Environment Variables** box to enable injection of configurations.
- Name this instance **sonar-maven-examples** as it will be referred to later.
- Use the [SonarQube URL](https://sonarcloud.io) but do **NOT** include a trailing slash or it will not work.
- Select the credentials you added in the previous step from the dropdown, i.e. *sonar-maven-examples*.
- Save the configuration.

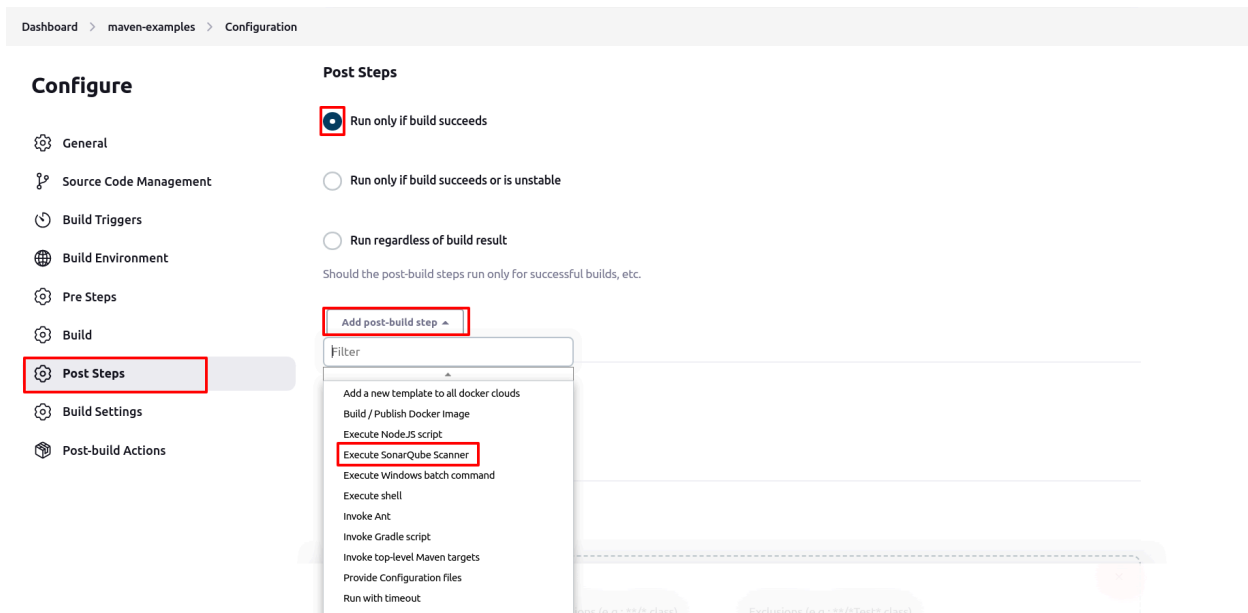
SonarQube installations
List of SonarQube installations



The screenshot shows the configuration form for a new SonarQube installation. The 'Name' field is filled with 'sonar-maven-examples' and is highlighted with a red box. The 'Server URL' field is filled with 'https://sonarcloud.io' and is also highlighted with a red box. The 'Server authentication token' dropdown menu is set to 'sonar-maven-examples' and is highlighted with a red box. Below the form, there is an '+ Add' button and an 'Advanced' dropdown menu.

Scanning Code and Reporting to SonarQube

In Jenkins, go to the configuration page for the **maven-examples** job that you created to use code coverage and add a post build step. Select **Execute Sonarqube Scanner**.



Put the following [snippet](#) into the **Analysis properties** text box:

```
# Required metadata
# Uncomment the following line and add your Org/Username from GitHub
or such
sonar.organization=YourGitHubOrg
# If using Sonar Cloud, Replace Project Key with the Actual

sonar.projectKey=yourGitHubOrg_ProjectName
sonar.projectName=Jacoco Code Coverage
sonar.projectVersion=1.0
# Comma-separated paths to directories with sources (required)
sonar.sources=code-coverage-jacoco/src/main
# Encoding of the source files
sonar.sourceEncoding=UTF-8
sonar.java.binaries=.
sonar.coverage.jacoco.xmlReportPaths=code-coverage-jacoco/target/site/
jacoco-ut/jacoco.xml
```

Dashboard > maven-examples > Configuration

Post Steps

Configure

- General
- Source Code Management
- Build Triggers
- Build Environment
- Pre Steps
- Build
- Post Steps**
- Build Settings
- Post-build Actions

☒ Run only if build succeeds

☐ Run only if build succeeds or is unstable

☐ Run regardless of build result

Should the post-build steps run only for successful builds, etc.

Execute SonarQube Scanner

Task to run ?

JDK ?

JDK to be used for this SonarQube analysis

(Inherit From Job)

Path to project properties ?

Analysis properties ?

```
# Required properties
# Uncomment the following line and add your Org/Username from GitHub or such
sonar.organization=eeganlf
# If using Sonar Cloud, Replace Project Key with the Actual
sonar.projectKey=eeganlf_maven-examples
sonar.projectName=Jacoco Code Coverage
```

Save Apply

Ensure you replace the properties for `sonar.organization` and `sonar.projectKey` with what you see in the **Sonar Cloud Analysis Configuration** page:

Execute the SonarScanner from your computer

Run the following command in your project's folder.

```
sonar-scanner \
-Dsonar.organization=eeganlf \
-Dsonar.projectKey=eeganlf_maven-examples \
-Dsonar.sources=. \
-Dsonar.host.url=https://sonarcloud.io
```

Copy

Please visit the [SonarScanner documentation](#) for more details.

Back in Jenkins, build the project. Click the SonarQube link to navigate to SonarQube:

Maven project maven-examples

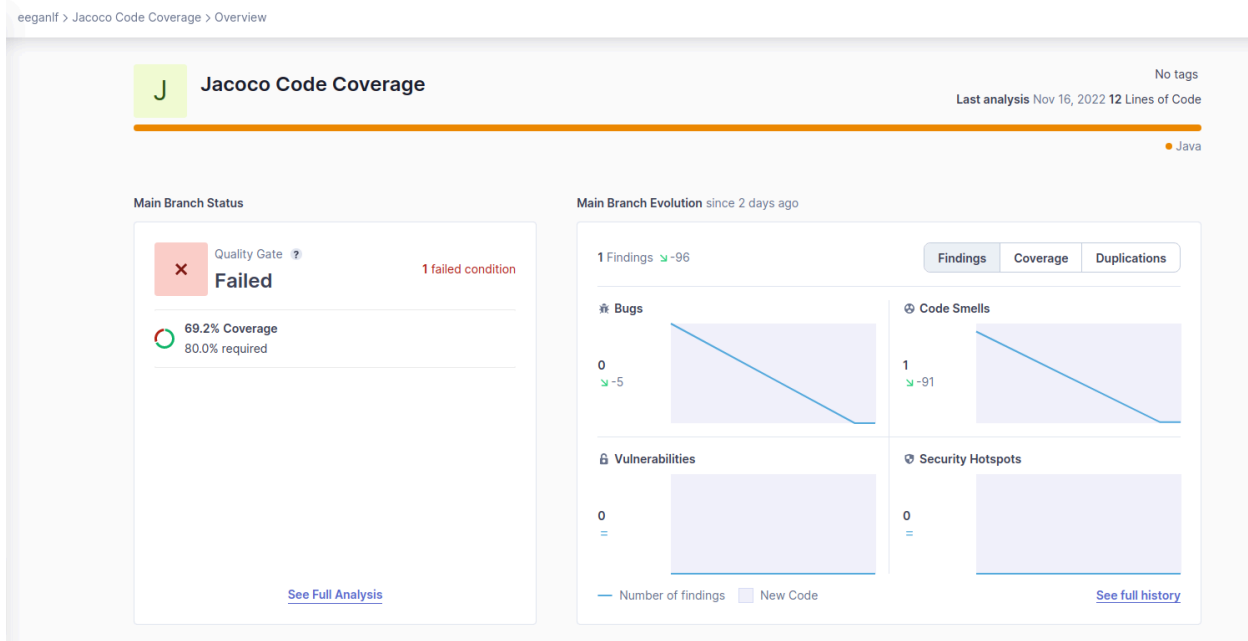


Latest Test Result (no failures)

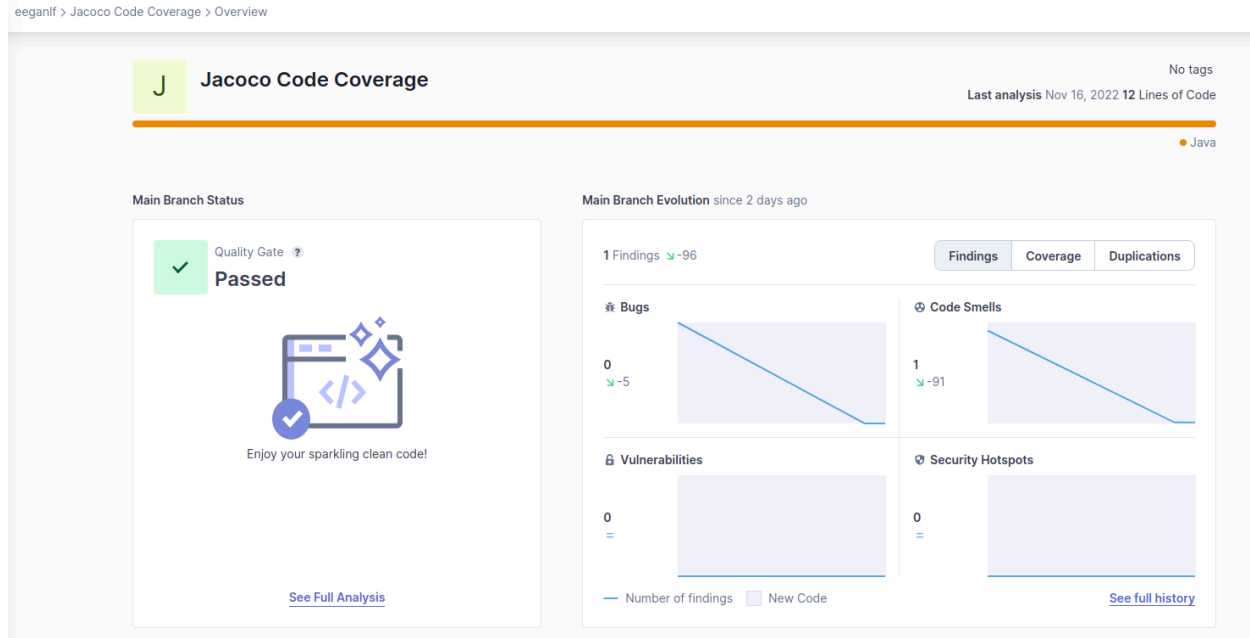
Permalinks

- [Last build \(#9\), 2 hr 5 min ago](#)
- [Last stable build \(#9\), 2 hr 5 min ago](#)
- [Last successful build \(#9\), 2 hr 5 min ago](#)
- [Last completed build \(#9\), 2 hr 5 min ago](#)

You will see that sonarcloud.io shows that the project failed and displays the reason why it failed. In this case, it failed because we have less than 80% test coverage.



Back on the GitHub repository for **maven-examples**, merge the **ut2** branch into the master and run **Build Now** in Jenkins on the **maven-examples** job. After it is complete, follow the **SonarQube** link. The job should now be displayed as passed:



Note that this does not keep Jenkins from successfully building the job. It is an illustration of how quality gates can be configured to output a pass or fail result for a particular job. We do, however, want to enable SonarQube to fail builds before they get to production. We can and should do this in the Jenkinsfile. We will accomplish this in the next section for the **example-voting-app**.

Adding SonarQube Scanner Stage to Jenkinsfile

You want SonarQube to fail the pipeline before the project is deployed if the code does not pass analysis. Add the following code snippet to the consolidated Jenkinsfile created earlier while setting up a monopipe. Place the code above the **stage ('deploy to dev')** block of code.

file: example-voting-app/Jenkinsfile

```
stage('Sonarqube') {
    agent any
    when{
        branch 'master'
    }
    environment{
```

```

        sonarpath = tool 'SonarScanner'
    }

    steps {
        echo 'Running Sonarqube Analysis..'
        withSonarQubeEnv('sonar-instavote') {
            sh "${sonarpath}/bin/sonar-scanner
-Dproject.settings=sonar-project.properties
-Dorg.jenkinsci.plugins.durabletask.BourneShellScript.HEARTBEAT_CHECK_
INTERVAL=86400"
        }
    }
}

stage("Quality Gate") {
    steps {
        timeout(time: 1, unit: 'HOURS') {
            // Parameter indicates whether to set pipeline to
UNSTABLE if Quality Gate fails
            // true = set pipeline to UNSTABLE, false = don't
waitForQualityGate abortPipeline: true
        }
    }
}

```

You can access this snippet [here](#).

Also update `sonar-project.properties` file at the root of `example-voting-app` repository with *your* organization and project key as per configured on SonarCloud.

File: `example-voting-app/sonar-project.properties`

```

# Uncomment and update Org matching your configurations on Sonarcloud
sonar.organization=your-org
sonar.projectKey=your-org_example-voting-app
sonar.projectName=LFS261 example voting app
sonar.projectVersion=1.0

# Comma-separated paths to directories with sources (required)

sonar.sources=worker

# Encoding of the source files

sonar.sourceEncoding=UTF-8

```

```
sonar.java.binaries=worker
```

```
sonar.coverage.jacoco.xmlReportPaths=worker/target
```

You also need to add Credentials to connect to the project added to SonarCloud. Navigate to **Jenkins > Manage Jenkins > Credentials > System > Global credentials (unrestricted)** and click **Add Credentials**:

Dashboard > Manage Jenkins > Credentials > System > Global credentials (unrestricted) >

Global credentials (unrestricted) + Add Credentials

Credentials that should be available irrespective of domain specification to requirements matching.

ID	Name	Kind	Description
0d47ee8b-691f-4e9e-8b9e-02049e396a56	Secret text	Secret text	
jenkins-github-access-token	jenkins-github-access-token	Secret text	
github-auth-token	eeganlf/***** (auth token for github)	Username with password	auth token for github
sonar-maven-examples	sonar-maven-examples	Secret text	sonar-maven-examples
dockerlogin	eeganlf/***** (dockerlogin)	Username with password	dockerlogin

Find the **SONAR_TOKEN** on the **LFS261-example-voting-app** configuration page and copy it:

eeganlf > LFS261-example-voting-app > Configure

Configure the SONAR_TOKEN environment variable

- 1 Name of the environment variable:
- 2 Value of the environment variable:

Paste this token into the **Secret** text box while configuring the credentials:

Dashboard > Manage Jenkins > Credentials > System > Global credentials (unrestricted) >

New credentials

Kind

Secret text

Scope ?

Global (Jenkins, nodes, items, all child items, etc)

Secret

ID ?

sonar-instavote

Description ?

sonar-instavote

Create

Create a new SonarQube Server configuration from **Jenkins > Manage Jenkins > System > SonarQube Servers**.

Name

sonar-instavote

Server URL

Default is http://localhost:9000

https://sonarcloud.io

Server authentication token

SonarQube authentication token. Mandatory when anonymous access is disabled.

sonar-instavote

Add

Advanced...

Add SonarQube

Save Apply

Remember to make sure that the `https://sonarcloud.io` does **NOT** have a trailing slash.

The free quality gate provided by SonarCloud only performs analysis on 20 lines or more of new code. Replace the entirety of the

`example-voting-app/worker/src/main/java/worker/Worker.java` file with this [code](#).

Commit to the master branch and observe the latest jobs being run with the next instance of the pipeline run. To view the build in the alternate BlueOcean interface, select the interface from the side menu of the master branch:

Dashboard > instavote > monopipe > master

Status

Changes

Build Now

View Configuration

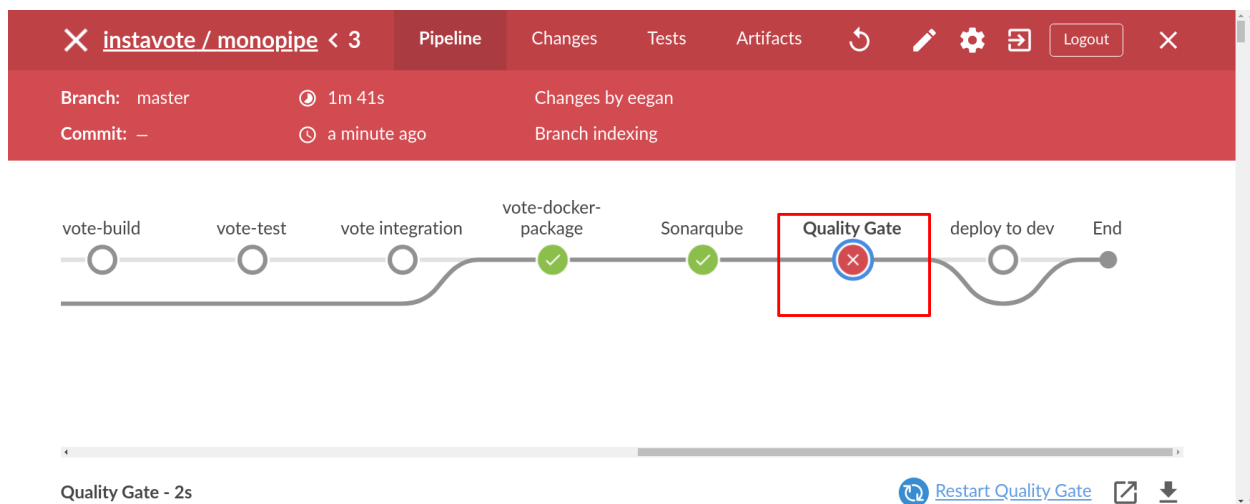
Full Stage View

Open Blue Ocean

GitHub

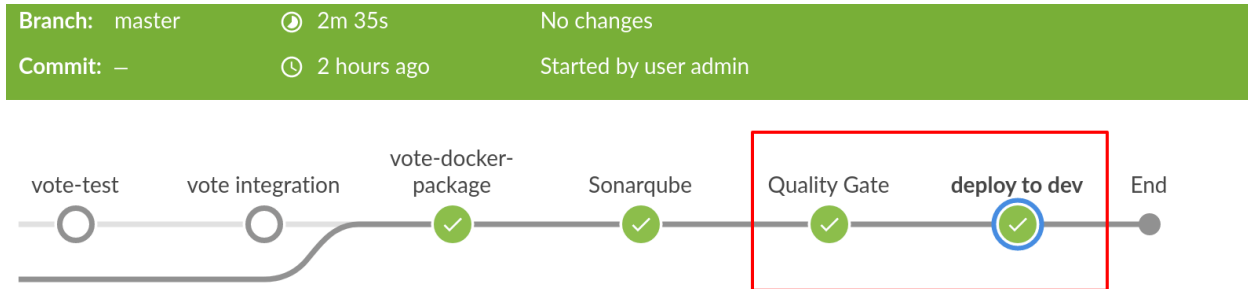
Pipeline Syntax

After the master branch builds in the monopipe job, you will notice that it fails at the **Quality Gate** step:



To get our worker app to pass the quality gate, we need to provide test coverage greater than 80%. Replace

`LFS261-example-voting-app/worker/src/test/java/worker/UnitWorker.java` with this [code](#).



Adding Integration Tests

Integration tests for the `vote` Python app are already added to the `vote/` subdirectory of the repository. To add integration tests to the pipeline, follow the steps below.

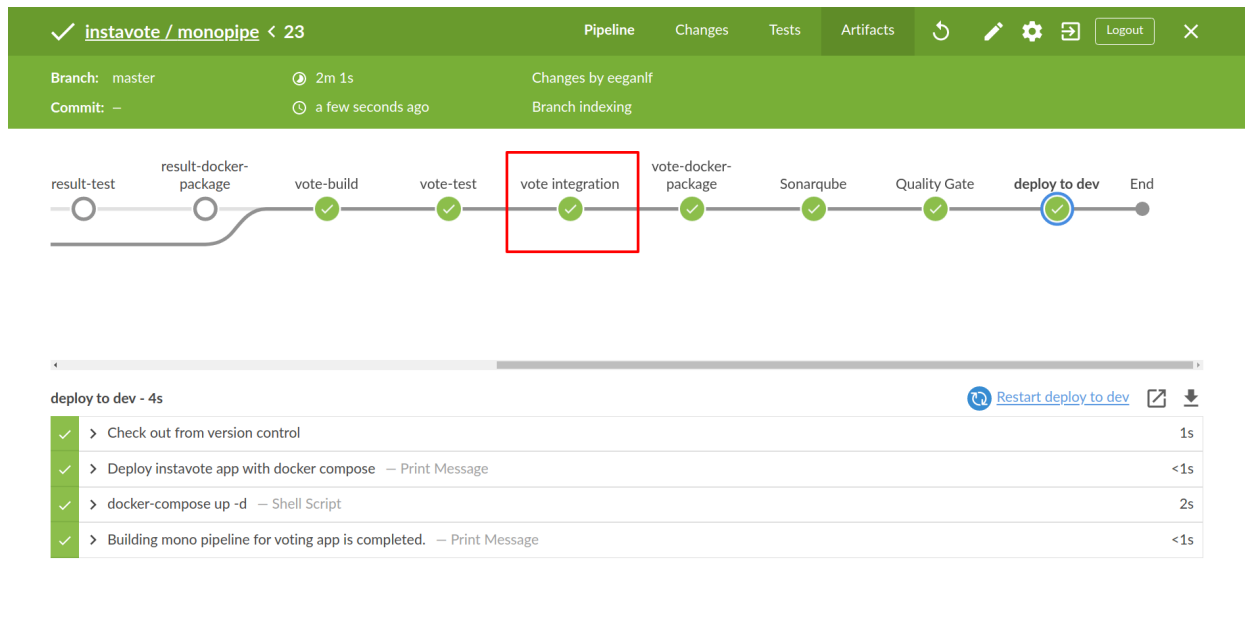
Examine the script at `vote/integration_test.sh` in the `example-voting-app` repository. Reading the script and the docker compose setup it uses should give you a good understanding of how integration tests are set up. Add a stage to the Jenkinsfile to run the integration tests as follows. Place the below code block just after the stage `vote-test` stage which runs unit tests for the `vote` app:

file: `example-voting-app/Jenkinsfile`

```
stage('vote integration'){
    agent any
    when{
        changeset '**/vote/**'
        branch 'master'
    }
    steps{
        echo 'Running Integration Tests on vote app'
        dir('vote'){
            sh 'sh integration_test.sh'
        }
    }
}
```

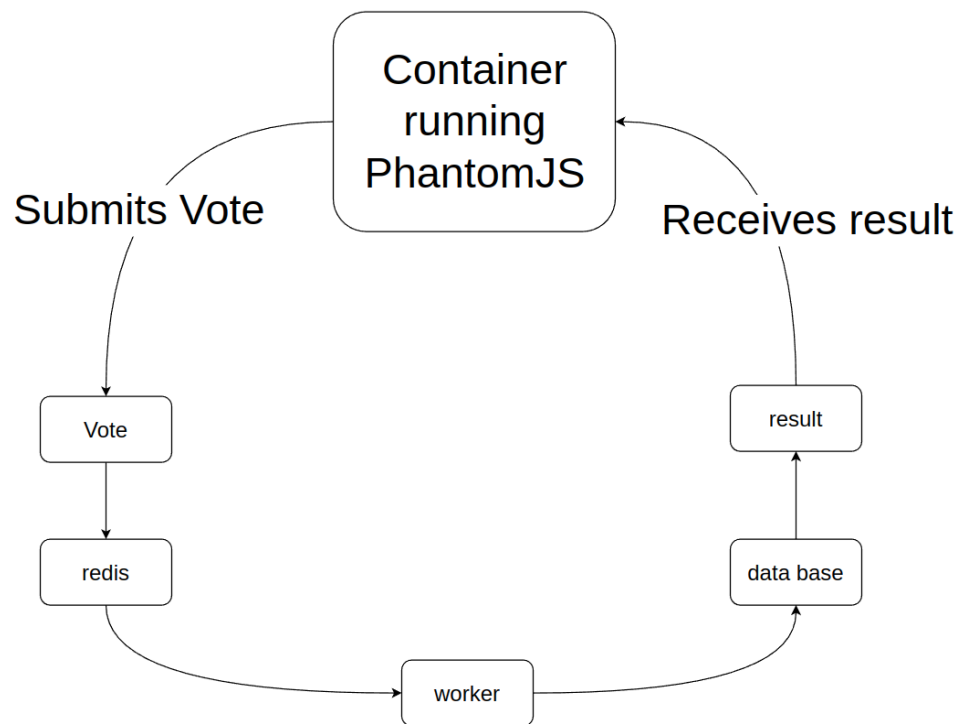
```
}
```

You should see a similar output in the Blue Ocean interface as the screenshot below. This indicates a successful execution of integration tests for the `vote` app:



E2E Tests

Examine the script `example-voting-app/e2e.sh` in the `example-voting-app` repository as well as the `example-voting-app/e2e/docker-compose.yml`. You do not need to understand all of the code, but reading the script and the Docker Compose setup it uses will give you a better understanding of how e2e tests are set up.



The .env File

`example-voting-app/e2e/docker-compose.yml` uses environment variables to decide which images to use to spin up the containers for the e2e tests.

Inside the `example-voting-app/e2e/.env` file you will find the following:

```
VOTE_IMAGE=xxxx/vote:latest
WORKER_IMAGE=xxxx/worker:latest
RESULT_IMAGE=xxxx/result:latest
```

Replace `xxxx` with your own Docker Hub user ID.

Setting Up E2E

Once the `.env` file is edited, run the following command from inside the `example-voting-app/e2e` which contains the `docker-compose.yml` file:

```
docker compose build
```


This will build the images necessary for the e2e test. It also ensures that any changes made to testing files are picked up by Docker Compose. After everything is built, run:

```
docker compose up -d
```

```
[+] Running 8/8
  ⚙ Network e2e_back-tier    Created           0.1s
  ⚙ Network e2e_front-tier   Created           0.1s
  ⚙ Container e2e-db-1        Started           1.8s
  ⚙ Container e2e-redis-1     Started           1.9s
  ⚙ Container e2e-worker-1    Started           2.8s
  ⚙ Container e2e-vote-1      Started           3.1s
  ⚙ Container e2e-result-1    Started           3.4s
  ⚙ Container e2e-e2e-1       Started           3.9s
```

To list your running containers to find their associated ports, run:

```
sudo docker ps
```

```
IMAGE                                PORTS
xxxxxxx/result:latest                0.0.0.0:49156->80/tcp, :::49156->80/tcp
xxxxxxx/vote:latest                  0.0.0.0:49155->80/tcp, :::49155->80/tcp
```

Find the containers associated with the `result:latest` image and the `vote:latest` image. Visit `IPADDRESS:VOTECONTAINERPORT` and `IPADDRESS:RESULTCONTAINERPORT`. Submit a vote. This is manual e2e testing.

To automate the process, we will now spin up the e2e service, which will automatically submit a vote for you. The e2e service is listed in the

`example-voting-app/e2e/docker-compose.yml`. Run:

```
docker compose run --rm e2e
```

```
-----
Current Votes Count: 13
-----
```

```
I: Submitting one more vote...
```

```
-----
New Votes Count: 14
-----
```

```
I: Checking if votes tally.....
```

```
-----  
Tests passed  
-----
```

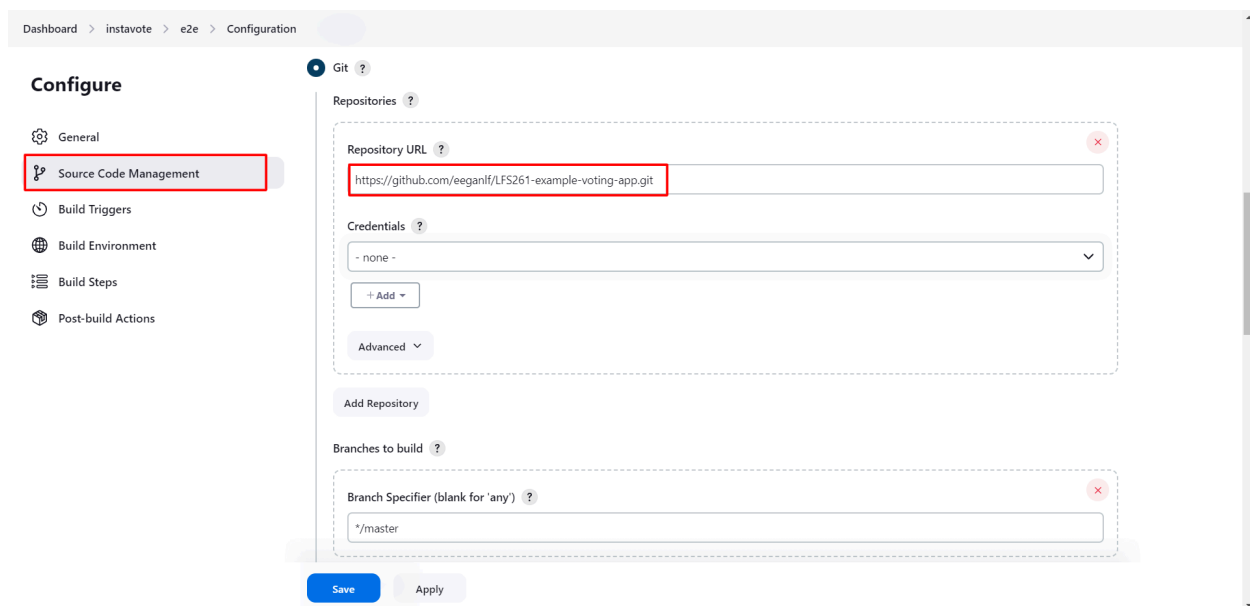
You should see something similar to the output above, though your counts will differ. If it fails, try running it again, and it should pass. This means that the application passed e2e testing. Shut the application down with the following command from inside `example-voting-app/e2e`:

```
docker compose down
```

Using a container to test our application is the first building block to integrating the e2e test into our CI process, which we will do next.

Integrating E2E Tests with Jenkins

First, create a freestyle project called e2e in Jenkins. Add the Git repository:



Inside the **Build Steps > Add build step > Execute shell > Command input** box paste:
`./e2e.sh`

≡ Execute shell ?

Command

See [the list of available environment variables](#)

```
./e2e.sh
```

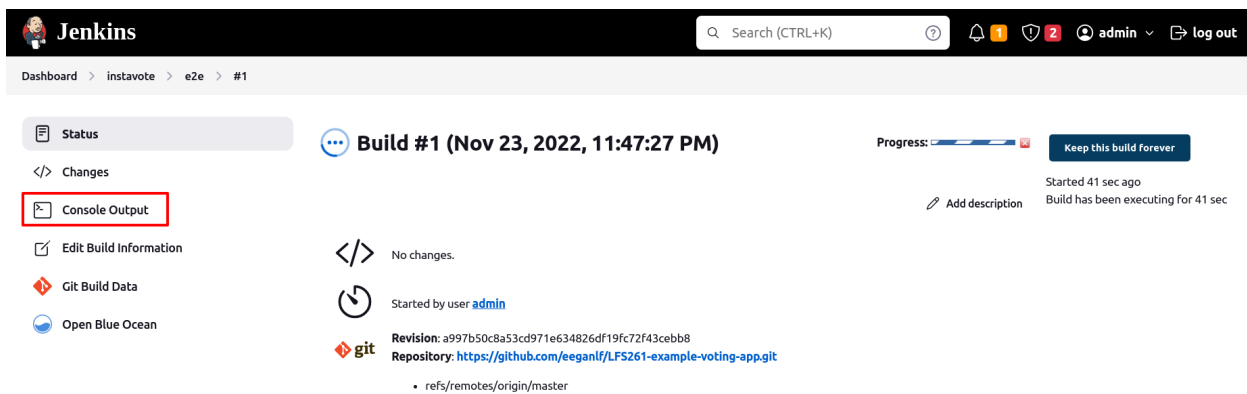
Advanced...

Save the configuration.

Run the job and click the build:

The screenshot shows the Jenkins web interface for a project named 'e2e'. The left sidebar contains a list of actions: Status, Changes, Workspace, Build Now (highlighted with a red box), Configure, Delete Project, Favorite, Move, Open Blue Ocean, and Rename. The main area displays 'Project e2e' with the full project name 'instavote/e2e'. Below this, there are 'Permalinks' and a 'Build History' section. The 'Build History' section shows a list of builds, with the first build (Build #1) highlighted with a red box. The build details for Build #1 show it was completed on Nov 23, 2022, at 11:47 PM, with a status of 'Success'.

Click **Console output** to see the console output:



The image shows the Jenkins web interface for a build. The top navigation bar includes the Jenkins logo, a search bar, and user information. The breadcrumb trail is Dashboard > instavote > e2e > #1. On the left sidebar, the 'Console Output' link is highlighted with a red rectangle. The main content area displays 'Build #1 (Nov 23, 2022, 11:47:27 PM)' with a progress bar and a 'Keep this build Forever' button. Below this, it shows 'No changes.' and 'Started by user admin'. The Git section indicates the revision and repository URL. The console output area is currently empty.

Jenkins

Search (CTRL+K)

Dashboard > instavote > e2e > #1

Status

Changes

Console Output

Edit Build Information

Git Build Data

Open Blue Ocean

Build #1 (Nov 23, 2022, 11:47:27 PM)

Progress: 

Keep this build Forever

Add description

Started 41 sec ago
Build has been executing for 41 sec

No changes.

Started by user [admin](#)

Revision: a997b50c8a53cd971e634826df19fc72f43cebb8
Repository: <https://github.com/eeganlf/LFS261-example-voting-app.git>

- refs/remotes/origin/master

If the tests fail, try running the job twice more. You should ultimately see something like the following:

```

Current Votes Count: 7
-----

I: Submitting one more vote...

-----

New Votes Count: 8
-----

I: Checking if votes tally.....

[92mTests passed

Stopping e2e_vote_1 ...
Stopping e2e_worker_1 ...
Stopping e2e_result_1 ...
Stopping e2e_redis_1 ...
Stopping e2e_db_1 ...
[2B[1A[2K
Stopping e2e_db_1 ... [32mdone[0m
[1BRemoving e2e_e2e_1 ...
Removing e2e_vote_1 ...
Removing e2e_worker_1 ...
Removing e2e_result_1 ...
Removing e2e_redis_1 ...
Removing e2e_db_1 ...
[1A[2K
Removing e2e_db_1 ... [32mdone[0m
[1B[2A[2K
Removing e2e_redis_1 ... [32mdone[0m
[2B[5A[2K
Removing e2e_vote_1 ... [32mdone[0m
[5B[6A[2K
Removing e2e_e2e_1 ... [32mdone[0m
[6B[3A[2K
Removing e2e_result_1 ... [32mdone[0m
[3B[4A[2K
Removing e2e_worker_1 ... [32mdone[0m
[4BRemoving network e2e_back-tier
Removing network e2e_front-tier
Finished: SUCCESS

```

You should see that the tests passed and then you should see the containers being cleaned up with a `SUCCESS` message at the bottom of the output. You have successfully run e2e tests with Jenkins.

Summary

In this lab, we integrated Jacoco into Jenkins to obtain automatic testing coverage analysis. We then integrated SonarQube into Jenkins for automatic code analysis. Finally, we added integration and e2e tests into Jenkins.